Подробные конспекты по операционным системам
Конспект 1: Управление процессами (Week 10)
Основные понятия
Процесс — это выполняющаяся программа, включающая код, данные, состояние процессора, память и ресурсы. Это динамическая сущность, в отличие от программы, которая является статическим набором инструкций на диске.
Разница между программой и процессом:

Программа — пассивный файл на диске с инструкциями
Процесс — активное выполнение программы с собственным контекстом (регистры, счетчик команд, память)
Одна программа может породить несколько независимых процессов

Структура процесса в памяти
Процесс организован в памяти следующим образом (снизу вверх):

Сегмент кода (Text Segment) — машинные инструкции программы, обычно доступны только для чтения
Сегмент данных (Data Segment) — глобальные и статические переменные с начальными значениями
BSS сегмент — неинициализированные глобальные переменные
Куча (Heap) — динамически выделяемая память (malloc, new), растет вверх
Стек (Stack) — локальные переменные функций, параметры, адреса возврата, растет вниз

Состояния процесса
Процесс в течение жизненного цикла переходит между следующими состояниями:

New (Новый) — процесс создается
Ready (Готовый) — ждет выделения процессорного времени
Running (Выполняющийся) — процесс выполняется на CPU
Waiting/Blocked (Ожидающий) — ждет завершения I/O операции или события
Terminated (Завершенный) — процесс закончил выполнение

Переходы между состояниями:

New → Ready: после инициализации
Ready → Running: планировщик выбрал процесс
Running → Waiting: процесс ждет I/O или спит
Waiting → Ready: I/O завершилось
Running → Terminated: выполнение завершено или процесс убит

Блок управления процессом (PCB)
PCB — центральная структура данных для управления процессом. Содержит:

Process ID (PID) — уникальный идентификатор
Состояние процесса — текущее состояние
Регистры CPU — значения всех регистров и счетчик команд
Информация о памяти — указатели на таблицы страниц, сегменты
Открытые файлы — дескрипторы файлов и I/O статус
Информация для планирования — приоритет, потраченное время CPU
Учетная информация — время создания, использование ресурсов

Переключение контекста
Context Switch — процесс сохранения состояния текущего процесса и загрузки состояния следующего процесса.
Шаги:

Сохранить состояние CPU текущего процесса в его PCB
Выбрать следующий процесс для выполнения
Загрузить состояние нового процесса из его PCB

Обновить регистры CPU, счетчик команд, MMU

Стоимость переключения контекста:

Временные затраты на сохранение/загрузку
Загрязнение кэша процессора
Промахи TLB (Translation Lookaside Buffer)

Планирование процессов
Типы планировщиков:

Долгосрочный планировщик — решает, какие задачи принять в систему
Среднесрочный планировщик — управляет подкачкой процессов в/из памяти
Краткосрочный планировщик — выбирает, какой процесс выполнять следующим

Цели планирования:

Утилизация CPU — максимально занять процессор
Пропускная способность — количество завершенных процессов за время
Время оборота (Turnaround Time) — время от прибытия до завершения
Время ожидания — время в очереди готовых
Время отклика — время до первого ответа

Алгоритмы планирования
1. FCFS (First-Come, First-Served)

Тип: невытесняющий
Принцип: очередь FIFO, процессы выполняются в порядке прибытия
Плюсы: простота, минимальное переключение контекста
Минусы: эффект конвоя (convoy effect) — короткие процессы ждут долгих

2. SJF (Shortest Job First) / SRTF (Shortest Remaining Time First)

SJF: невытесняющий, выбирает самый короткий процесс
SRTF: вытесняющий, прерывает текущий процесс для более короткого
Плюсы: оптимален для минимизации среднего времени ожидания
Минусы: требует предсказания времени выполнения, риск голодания длинных процессов

3. Priority Scheduling (Приоритетное планирование)

Каждый процесс имеет приоритет
Планировщик выбирает процесс с наивысшим приоритетом
Может быть вытесняющим или невытесняющим
Проблема: голодание низкоприоритетных процессов
Решение: aging (старение) — постепенное повышение приоритета ожидающих процессов

4. Round Robin (Карусель)

Тип: вытесняющий
Принцип: каждому процессу выделяется фиксированный квант времени
Процессы: организованы в круговую очередь
Плюсы: гарантирует ограниченное время ожидания, справедливость
Минусы: выбор кванта критичен (слишком малый → большие накладные расходы, слишком большой → плохое время отклика)
Применение: интерактивные системы

5. Multilevel Queue (Многоуровневая очередь)

Процессы делятся по типам: интерактивные, пакетные, системные
Каждая очередь имеет свой алгоритм планирования

Очереди с высоким приоритетом обслуживаются первыми
Риск: голодание низкоприоритетных очередей

6. Multilevel Feedback Queue (MLFQ)

Адаптивный алгоритм: процессы перемещаются между очередями
CPU-интенсивные процессы понижаются в приоритете
I/O-интенсивные остаются в высокоприоритетных очередях
Старение предотвращает голодание
Комбинирует преимущества Round Robin и приоритетного планирования

Проблемы планирования

Голодание (Starvation) — низкоприоритетные процессы никогда не выполняются
Инверсия приоритетов — высокоприоритетный процесс ждет низкоприоритетный
Большие накладные расходы — частое переключение контекста при малом кванте
Взрывной характер нагрузки — снижает справедливость и пропускную способность

Конспект 2: Управление памятью (Week 11)
Введение в управление памятью
Цели управления памятью:

Выделение памяти процессам
Защита памяти одних процессов от других
Совместное использование памяти
Поддержка виртуальной памяти
Обеспечение производительности

Адресные пространства
Логический адрес — адрес, генерируемый процессором (виртуальный адрес)
Физический адрес — реальное расположение в RAM
MMU (Memory Management Unit) — аппаратный блок, транслирующий логические адреса в физические.
Это обеспечивает изоляцию процессов и гибкость размещения в памяти.
Непрерывное распределение памяти
Ранние ОС использовали непрерывные блоки памяти:
Методы:

Фиксированные разделы — память делится на фиксированные блоки
Динамические разделы — размер блока определяется по запросу

Проблемы:

Фрагментация
Плохая утилизация памяти

Фрагментация
Внутренняя фрагментация — неиспользуемое пространство внутри выделенного блока (например, процессу нужно 18KB, выделили 20KB → 2KB потрачено впустую)
Внешняя фрагментация — множество мелких свободных блоков, разбросанных по памяти, которые в сумме достаточны, но не могут быть использованы из-за разрозненности
Непрерывные системы особенно подвержены фрагментации.
Страничная организация памяти (Paging)
Принцип:

Физическая память делится на фреймы (frames) фиксированного размера (обычно 4KB)
Логическая память процесса делится на страницы (pages) того же размера
Страницы могут загружаться в любые свободные фреймы

Преимущества:

Устраняет внешнюю фрагментацию
Упрощает выделение памяти
Позволяет загружать процесс не целиком

Таблица страниц — структура данных, отображающая логические страницы на физические фреймы.
Таблица страниц
Для каждого процесса ОС ведет таблицу страниц:
Содержит:

Номер фрейма для каждой страницы
Valid bit — страница находится в памяти или нет
Dirty bit — страница была изменена
Reference bit — к странице обращались недавно
Protection bits — права доступа (read/write/execute)

Проблема: таблица страниц хранится в памяти, доступ к ней замедляет работу.
TLB (Translation Lookaside Buffer)
TLB — аппаратный кэш недавних трансляций адресов.
Работа:

CPU генерирует логический адрес
Проверяется TLB
TLB hit — быстрая трансляция найдена
TLB miss — обращение к таблице страниц в памяти, результат кэшируется в TLB

TLB критичен для производительности, так как уменьшает накладные расходы на трансляцию адресов.
Сегментация
Сегментация делит память на логические единицы:

Сегмент кода
Сегмент данных
Сегмент стека

Особенности:

Переменные размеры сегментов
Защищает различные области программы
Позволяет осмысленный контроль доступа

Страничная организация vs Сегментация
КритерийСтраничная организацияСегментацияРазмерФиксированныйПеременныйФрагментацияНет внешнейВозможна внешняяЛогикаНевидима для программистаСоответствует логике программыЗащитаПо страницамПо сегментам
Многие современные ОС комбинируют оба подхода.
Виртуальная память
Концепция: позволяет процессам быть больше физической памяти, используя диск как расширение RAM.
Механизм:

Не все страницы процесса находятся в памяти одновременно
Страницы подкачиваются с диска по мере необходимости
Неиспользуемые страницы выгружаются на диск

Demand Paging (Страничная подкачка по требованию)
Принцип: страницы загружаются только при обращении к ним.
Преимущества:

Уменьшает использование памяти
Позволяет запускать больше процессов
Быстрый запуск программ

Page Fault (Ошибка страницы) — возникает при обращении к странице, которой нет в памяти.
Обработка Page Fault
Шаги:

CPU пытается обратиться к странице
Обнаруживается, что страница не в памяти (valid bit = 0)
Генерируется прерывание page fault
ОС приостанавливает процесс
ОС находит страницу на диске
Если RAM полна → выбирается страница-жертва для вытеснения
Если страница-жертва изменена (dirty bit = 1) → записывается на диск
Нужная страница загружается в освободившийся фрейм
Обновляется таблица страниц
Процесс возобновляется

Алгоритмы замещения страниц
Когда память полна, нужно выбрать страницу для вытеснения:
1. FIFO (First-In-First-Out)

Вытесняется самая старая загруженная страница
Плюс: простота реализации
Минус: страдает от аномалии Белади (больше фреймов → больше page faults в некоторых случаях)

2. Optimal (Оптимальный)

Вытесняется страница, которая не понадобится дольше всего
Идеален теоретически, но требует знания будущих обращений
Используется для сравнения других алгоритмов

3. LRU (Least Recently Used)

Вытесняется страница, к которой дольше всего не обращались
Хорошее приближение к оптимальному
Минус: требует отслеживания времени использования (дорого)
Реализация: стек страниц или счетчики

4. LFU (Least Frequently Used)

Вытесняется страница с наименьшим числом обращений
Учитывает частоту, а не давность

5. Random (Случайный)

Выбирается случайная страница
Прост, но непредсказуем

Производительность виртуальной памяти
Критический фактор: частота page faults.
Эффективное время доступа = (1 - p) × время доступа к памяти + p × время обработки page fault
где p — вероятность page fault
Thrashing (Пробуксовка):

Система тратит больше времени на подкачку страниц, чем на выполнение процессов
Возникает при нехватке памяти
CPU простаивает в ожидании I/O

Решение: уменьшить степень многозадачности, увеличить память

Конспект 3: Файловые системы (Week 12)
Введение в файловые системы
Файловая система — компонент ОС для управления хранением данных на дисках.
Функции:

Организация файлов и каталогов
Управление метаданными
Контроль доступа
Обеспечение постоянства данных

Ответственность файловой системы

Создание, удаление, именование файлов
Построение иерархии каталогов
Отслеживание выделения и свободного места
Обеспечение безопасности: права, владение
Поддержание консистентности после сбоев

Атрибуты файлов и метаданные
Основные атрибуты:

Имя файла
Тип (расширение)
Расположение на диске
Размер

Временные метки:

Время создания
Последнего доступа
Последнего изменения

Владение и права:

Владелец (user)
Группа (group)
Права доступа (permissions)

Дополнительные флаги:

Hidden (скрытый)
System (системный)
Append-only (только добавление)
Immutable (неизменяемый)

Операции с файлами
Базовые операции:

create — создать файл
open — открыть для операций
close — закрыть файл
read — прочитать данные
write — записать данные
append — добавить в конец
seek — переместить указатель позиции (произвольный доступ)
truncate — обрезать файл

delete — удалить файл

Операции с правами:

chmod — изменить права доступа
chown — изменить владельца
chgrp — изменить группу

Операции с каталогами:

mkdir — создать каталог
rmdir — удалить каталог
mv — переместить/переименовать
cp — копировать

Структуры каталогов

Одноуровневый каталог — все файлы в одном каталоге (примитивно, проблемы с именами)
Двухуровневый каталог — отдельный каталог для каждого пользователя
Древовидный каталог (иерархический) — современный стандарт, файлы организованы в дерево каталогов
DAG (направленный ациклический граф) — позволяет разделяемые подкаталоги, символьные ссылки

Записи каталога хранят имена файлов + ссылки на метаданные.
Методы размещения файлов
1. Непрерывное размещение (Contiguous Allocation)

Файл хранится в последовательных блоках диска
Плюсы: быстрый последовательный и произвольный доступ
Минусы: внешняя фрагментация, сложность изменения размера

2. Связное размещение (Linked Allocation)

Каждый блок содержит указатель на следующий блок
Плюсы: легкое расширение, нет фрагментации
Минусы: медленный произвольный доступ, накладные расходы на указатели

3. Индексное размещение (Indexed Allocation)

Индексный блок хранит адреса всех блоков файла
Плюсы: быстрый прямой доступ, поддержка больших файлов через многоуровневые индексы
Минусы: накладные расходы на индексные блоки

Современные FS часто используют гибридные схемы (например, ext2, NTFS).
Управление свободным пространством
Методы отслеживания свободных блоков:

Битовая карта (Bitmap)

0 = свободен, 1 = занят
Компактна, быстрый поиск

Список свободных блоков

Связный список свободных блоков
Простота добавления/удаления

Группировка и подсчет

Хранение диапазонов свободных блоков

Монтирование и иерархия ФС
Монтирование — присоединение файловой системы к дереву каталогов.

Каждый раздел может иметь свой тип ФС
В Linux: иерархия начинается с корня /

/home — домашние каталоги пользователей
/etc — конфигурационные файлы
/var — изменяемые данные (логи, временные файлы)
/usr — программы и библиотеки

Модель прав доступа в Linux
Три категории владения:

User (владелец) — создатель файла
Group (группа) — группа пользователей
Others (остальные) — все прочие

Три типа прав:

r (read) — чтение
w (write) — запись
x (execute) — выполнение

Пример: -rwxr-xr--

Тип: - (обычный файл)
User: rwx (чтение, запись, выполнение)
Group: r-x (чтение, выполнение)
Others: r-- (только чтение)

chmod изменяет права:

Символьный режим: chmod u+x file (добавить выполнение владельцу)
Числовой режим: chmod 755 file (rwxr-xr-x)

Специальные биты:

SUID — выполнение с правами владельца файла
SGID — выполнение с правами группы / наследование группы в каталоге
Sticky bit — только владелец может удалять файлы в каталоге (например, /tmp)

Индексные дескрипторы (Inodes)
Inode — структура метаданных в Unix/Linux (не хранит имя файла).
Содержит:

Размер файла
Временные метки
Указатели на блоки данных
Права доступа и владельца
Счетчик жестких ссылок

Записи каталога отображают имена файлов на номера inode.
Жесткие ссылки — несколько имен указывают на один inode.

Журналирование и восстановление
Журналирование (Journaling) — запись изменений метаданных в журнал перед их применением.
Преимущества:

Быстрое восстановление после сбоя
Меньше риска повреждения ФС

Режимы журналирования:

Metadata-only — журналируются только метаданные (быстрее)
Full-data journaling — журналируются данные и метаданные (надежнее, но медленнее)

Примеры: ext4, NTFS используют журналирование.
Распространенные файловые системы
ext4 (Fourth Extended File System):

ФС по умолчанию в Linux
Журналирование
Экстенты (непрерывные блоки для больших файлов)
Поддержка больших файлов и разделов

NTFS (New Technology File System):

ФС Windows
MFT (Master File Table)
Расширенные права доступа
Журналирование
Сжатие и шифрование

FAT32/exFAT:

Простые, переносимые
Широко поддерживаются (USB флешки, камеры)
Ограничения на размер файлов (FAT32: 4GB)

XFS/Btrfs:

Высокопроизводительные
Снимки файловой системы (snapshots)
Масштабируемость

Факторы производительности ФС

Фрагментация — разброс блоков файла по диску замедляет доступ
Размер блока — компромисс между внутренней фрагментацией и производительностью
Паттерны доступа:

Последовательный доступ — быстрее
Произвольный доступ — медленнее

Кэширование и буферы — ОС кэширует часто используемые данные в RAM

Безопасность файловой системы

ACL (Access Control Lists) — расширенные списки контроля доступа
Шифрование at rest — защита данных на диске
Проверки целостности — контрольные суммы для обнаружения повреждений
Защита системных файлов — предотвращение изменения критических файлов

Конспект 4: Синхронизация и дедлоки (Week 13)
Понимание конкурентности
Конкурентность (Concurrency) — одновременное выполнение нескольких процессов/потоков.
Характеристики:

Процессы разделяют ядра CPU, память, файлы, устройства
Выполнение может чередоваться непредсказуемо
Решения планировщика влияют на порядок выполнения
Приводит к недетерминированному поведению

Зачем нужна синхронизация?
Ключевые причины:

Предотвращение гонки данных (race conditions)
Избежание потерянных обновлений
Гарантия консистентности данных
Обеспечение порядка зависимых операций
Защита памяти и разделяемых переменных
Обеспечение предсказуемого поведения программы

Проблема критической секции
Критическая секция — участок кода, обращающийся к разделяемым данным.
Требования:

Только один процесс может находиться в критической секции одновременно
Остальные должны ждать
После выхода другой процесс может войти

Три требования к корректным решениям
✓ Взаимное исключение (Mutual Exclusion)

Только один процесс внутри критической секции одновременно

✓ Прогресс (Progress)

Процесс не в критической секции не должен блокировать вход других
Решение о входе должно приниматься за конечное время

✓ Ограниченное ожидание (Bounded Waiting)

Каждый процесс в итоге получает доступ
Не допускается голодание

Аппаратные примитивы синхронизации
Низкоуровневые атомарные операции:
Test-and-Set (TSL)

Атомарно читает значение и устанавливает его в 1
Используется для реализации спинлоков

Compare-and-Swap (CAS)

Атомарно сравнивает значение и обменивает его при совпадении
Основа многих lock-free алгоритмов

Fetch-and-Add

Атомарно увеличивает значение и возвращает старое

Отключение прерываний

Работает только на однопроцессорных системах
Опасно, может привести к зависанию

Программные инструменты синхронизации
1. Spinlocks (Спинлоки)

Активное ожидание: процесс крутится в цикле, проверяя блокировку
Эффективны для коротких критических секций
Тратят процессорное время впустую

2. Mutex Locks (Мьютексы)

Двоичное состояние: заблокирован/разблокирован
Защищают критические секции
Поток должен захватить блокировку перед входом
Остальные ждут или блокируются
ОС может переключить контекст на другой поток при недоступности

Использование:
acquire(mutex);
// критическая секция
release(mutex);
3. Семафоры (Semaphores)
Типы:

Бинарный семафор — работает как мьютекс (0 или 1)
Счетный семафор — управляет несколькими идентичными ресурсами (N экземпляров)

Операции:

- **wait() / P()** — уменьшить счетчик; если < 0, блокировать процесс
- **signal() / V()** — увеличить счетчик; разбудить ожидающий процесс

Пример:

```
semaphore mutex = 1;

wait(mutex);
// критическая секция
signal(mutex);
```

4. Мониторы и условные переменные

Мониторы предоставляют:

- Неявное взаимное исключение
- Условные переменные для ожидания
- Структурированную синхронизацию внутри одного объекта/модуля

Условные переменные:

- `wait()` — освободить блокировку и ждать сигнала
- `signal()` / `notify()` — разбудить один ожидающий поток
- `broadcast()` / `notifyAll()` — разбудить все ожидающие потоки

Примеры: `synchronized` в Java, `Lock` в Python

Классические проблемы синхронизации

Проблема производителя-потребителя (Producer-Consumer)

Описание:

- Производитель генерирует данные
- Потребитель использует данные
- Разделяемый буфер ограниченного размера

Проблемы:

- Переполнение буфера (producer опережает)
- Опустошение буфера (consumer опережает)

Решение:

```
semaphore full = 0;    // счетчик заполненных слотов
semaphore empty = N;   // счетчик пустых слотов
semaphore mutex = 1;   // защита буфера

Producer:
  wait(empty);
  wait(mutex);
  // добавить в буфер
  signal(mutex);
  signal(full);

Consumer:
  wait(full);
  wait(mutex);
  // извлечь из буфера
  signal(mutex);
  signal(empty);
```

Проблема читателей-писателей (Readers-Writers)

Правила:

- Несколько читателей могут читать одновременно
- Писатель должен иметь эксклюзивный доступ
- Читатели не должны голодать писателя
- Писатели не должны голодать читателей

Варианты решения:

- **Приоритет читателям** — писатели могут голодать
- **Приоритет писателям** — читатели могут голодать
- **Справедливое решение** — баланс между двумя

Проблема обедающих философов (Dining Philosophers)

Моделирует:

- Конкуренцию за ресурсы
- Ограниченные разделяемые ресурсы, вызывающие дедлок
- Необходимость синхронизированной стратегии

Описание: N философов сидят за круглым столом, между каждыми двумя одна вилка. Философ может

есть только если взял обе вилки (слева и справа).

Проблема: все философы одновременно возьмут левую вилку → дедлок.

Решения:

- Ограничить количество одновременно обедающих
- Нечетные философы берут сначала левую вилку, четные — правую
- Использовать семафор для контроля доступа

Что такое дедлок (Deadlock)?

Определение: набор процессов находится в состоянии постоянной блокировки, ожидая ресурсов, удерживаемых друг другом.

Характеристики:

- Ни один процесс не может продолжить выполнение
- Круговое ожидание
- Все ждут, никто не может освободить ресурсы

Четыре условия дедлока (Условия Коффмана)

Дедлок возможен только если выполняются **все четыре условия**:

1. **Взаимное исключение (Mutual Exclusion)**
   - Ресурс удерживается только одним процессом одновременно
2. **Удержание и ожидание (Hold and Wait)**
   - Процесс удерживает один ресурс и ждет другой
3. **Отсутствие вытеснения (No Preemption)**
   - Ресурсы не могут быть принудительно отняты
4. **Круговое ожидание (Circular Wait)**
   - Существует цикл процессов, каждый ждет ресурса от следующего

Граф распределения ресурсов (RAG)

Компоненты:

- **Процессы** — круги
- **Ресурсы** — квадраты (с точками внутри = экземпляры ресурса)
- **Ребра запроса** — стрелка от процесса к ресурсу
- **Ребра назначения** — стрелка от ресурса к процессу

Анализ:

- **Цикл в графе** указывает на возможный дедлок (в системах с одним экземпляром ресурса — гарантированный дедлок)
- **Отсутствие цикла** — дедлока нет

Предотвращение дедлоков

Нарушить одно из четырех условий Коффмана:

1. Устранить взаимное исключение

- Сделать ресурсы разделяемыми где возможно
- Пример: файлы только для чтения, разделяемая память с блокировками

2. Устранить удержание и ожидание

- Требовать, чтобы процесс запрашивал все ресурсы сразу
- Или заставлять процесс освобождать ресурсы перед запросом новых
- **Минус:** низкая утилизация ресурсов, голодание

3. Разрешить вытеснение

- ОС принудительно забирает ресурсы у процесса
- Особенно полезно для CPU, памяти
- Сложно для некоторых типов ресурсов (например, принтеры)

4. Нарушить круговое ожидание

- Ввести строгий порядок запроса ресурсов
- Все процессы должны запрашивать ресурсы в предопределенном порядке
- Пример: ресурсы пронумерованы, процесс может запросить ресурс N только если не держит ресурс с номером > N

Избежание дедлоков (Banker's Algorithm)

Концепция: система проверяет, останется ли система в безопасном состоянии после выделения ресурса.

Шаги:

1. Процесс объявляет максимальные потребности в ресурсах
2. ОС симулирует выделение
3. Если система остается **безопасной** → выделить ресурс
4. Если система становится **небезопасной** → отклонить запрос

Безопасное состояние: существует последовательность выполнения процессов, при которой все могут завершиться.

Минусы:

- Требует знания максимальных потребностей заранее
- Консервативен (может отклонить безопасные запросы)
- Накладные расходы на проверку

Обнаружение и восстановление от дедлоков

Обнаружение:

- ОС периодически запускает алгоритм поиска циклов в графе распределения ресурсов
- Если цикл найден → дедлок существует

Восстановление:

1. **Завершение процессов:**
    - Завершить все процессы в дедлоке
    - Завершать по одному, пока дедлок не исчезнет
2. **Откат к безопасной точке:**
    - Использовать checkpoints
    - Откатить один или несколько процессов
3. **Вытеснение ресурсов:**
    - Забрать ресурсы у процессов

    - Выбрать жертву (обычно процесс с наименьшими затратами)
4. **Убить процесс с наименьшим приоритетом или наименьшей стоимостью**

Критерии выбора жертвы:

- Приоритет процесса
- Время выполнения
- Количество используемых ресурсов
- Количество работы, которую еще предстоит выполнить

Конспект 5: Безопасность и защита (Week 14)

Что такое безопасность в ОС?

Цели безопасности:

- **Конфиденциальность (Confidentiality)** — предотвращение несанкционированного доступа к данным
- **Целостность (Integrity)** — предотвращение несанкционированного изменения
- **Доступность (Availability)** — обеспечение доступности ресурсов
- **Аутентичность (Authenticity)** — проверка подлинности идентификаторов
- **Подотчетность (Accountability)** — отслеживание действий пользователей

Защита vs Безопасность

Защита (Protection):

- Внутренние механизмы ОС
- Контролирует доступ процессов к ресурсам
- Предотвращает случайное вмешательство

Безопасность (Security):

- Защита от внешних угроз
- Предотвращает преднамеренное неправомерное использование
- Включает аутентификацию и шифрование

Угрозы безопасности ОС

Распространенные угрозы:

- **Malware (вредоносное ПО)** — вирусы, черви, трояны
- **Несанкционированный доступ** — взлом учетных записей
- **Повышение привилегий** — получение root/admin прав
- **Атаки на пароли** — brute force, словарные атаки
- **Отказ в обслуживании (DoS)** — перегрузка системы
- **Внутренние угрозы** — злонамеренные пользователи внутри организации
- **Социальная инженерия** — фишинг, обман пользователей

Основы контроля доступа

Ключевые компоненты:

- **Субъекты (Subjects)** — пользователи, процессы
- **Объекты (Objects)** — файлы, память, устройства
- **Права (Rights)** — чтение, запись, выполнение, удаление
- **Механизм принуждения** — ОС решает, кто может получить доступ

Модели контроля доступа

1. DAC (Discretionary Access Control) — Дискреционное управление доступом

Характеристики:

- Владельцы файлов решают, кому предоставить доступ
- Гибкий, но менее безопасный
- Уязвим к злоупотреблению привилегиями
- Используется в Windows и классическом Unix

Пример: владелец файла может дать права любому пользователю

2. MAC (Mandatory Access Control) — Мандатное управление доступом

Характеристики:

- Навязывается системой, а не пользователями
- Метки уровней безопасности (Top Secret, Confidential, Public)
- Пользователь не может изменить классификацию
- Используется в правительственных системах, SELinux, AppArmor

Пример: файл с меткой "Secret" недоступен пользователям с уровнем "Confidential"

3. RBAC (Role-Based Access Control) — Ролевое управление доступом

Принципы:

- Права группируются в роли
- Пользователи назначаются на роли
- Очень масштабируемо для организаций

Примеры ролей:

- Administrator — полный доступ
- Auditor — только чтение логов
- Developer — доступ к коду и средам разработки
- Student/Teacher — академические права

4. ABAC (Attribute-Based Access Control) — Управление доступом на основе атрибутов

- Решения о доступе основаны на атрибутах субъекта, объекта, окружения
- Очень гибкий и детальный контроль

Методы аутентификации

Распространенные методы:

1. **Парольная аутентификация**
    - Самый распространенный метод
    - Требует политики надежных паролей
2. **Двухфакторная аутентификация (2FA)**
    - Комбинация: что-то знаешь + что-то имеешь
    - Пример: пароль + SMS код
3. **Биометрия**
    - Отпечаток пальца
    - Face ID
    - Сканирование сетчатки

4. **Токены и смарт-карты**
    - Физические устройства для аутентификации
5. **OAuth, SSO, федеративная идентификация**
    - Вход через Google, Facebook
    - Единый вход для множества сервисов

Управление пользователями и группами

Включает:

- Создание/удаление учетных записей
- Назначение групп
- Управление привилегиями
- Применение политик паролей
- Контроль поведения при входе в систему

Группы:

- Упрощают управление правами
- Пользователь может принадлежать нескольким группам
- Права назначаются группам, а не отдельным пользователям

Защита файловой системы

Механизмы:

1. **Права доступа к файлам (r/w/x)**
    - Основной механизм в Unix/Linux
2. **ACL (Access Control Lists)**
    - Расширенные права доступа
    - Более детальный контроль, чем стандартные права
3. **Шифрование**
    - Защита данных в состоянии покоя
    - Примеры: LUKS, BitLocker, FileVault
4. **Мониторинг целостности файлов**
    - Обнаружение несанкционированных изменений
    - Использование контрольных сумм
5. **Специальные биты (SUID/SGID)**
    - SUID: запуск с правами владельца
    - SGID: запуск с правами группы

Механизмы защиты памяти

Инструменты:

- **Базовые и граничные регистры**
    - Ограничивают диапазон адресов процесса
- **Страничная организация и сегментация**
    - Изоляция адресных пространств процессов
- **Права Read/Write/Execute**
    - Контроль доступа к страницам памяти
- **Разделение режимов kernel/user**
    - Критический код в привилегированном режиме
- **Изоляция адресного пространства**
    - Процессы не могут обращаться к памяти друг друга

Аппаратная защита

Механизмы:

- **Режимы CPU (user vs kernel)**
  - Двухуровневая защита
  - Привилегированные инструкции только в kernel mode
- **Привилегированные инструкции**
  - Управление I/O, изменение таблиц страниц доступны только ядру
- **Защита I/O**
  - Пользовательские процессы не могут напрямую обращаться к устройствам
- **Прерывания таймера**
  - Предотвращение монополизации CPU одним процессом

Контроль доступа к системе и права

Ключевые элементы:

- **Права доступа к файлам**
- **Привилегии на уровне процессов**
- **Политики сетевого доступа** (firewalls, iptables)
- **Правила доступа к устройствам**
- **Принцип наименьших привилегий**
  - Процесс должен иметь только минимально необходимые права
  - Уменьшает потенциальный ущерб от компрометации

Аудит и логирование

Цели:

- Отслеживание действий пользователей
- Обнаружение подозрительного поведения
- Поддержка расследований
- Генерация оповещений о безопасности

Что логировать:

- Попытки входа (успешные и неудачные)
- Изменения прав и конфигураций
- Доступ к критическим файлам
- Выполнение привилегированных команд

Инструменты в Linux:

- `syslog`, `rsyslog`
- `/var/log/` — каталог с логами
- `auditd` — подсистема аудита

Распространенные атаки безопасности

Примеры:

1. **Buffer overflow (переполнение буфера)**
   - Запись за пределы выделенной памяти
   - Может привести к выполнению произвольного кода
2. **SQL injection (SQL-инъекция)**
   - Внедрение вредоносного SQL кода
   - Компрометация баз данных
3. **Cross-site scripting (XSS)**
   - Внедрение скриптов в веб-страницы

    - Кража сессий пользователей
4. **Man-in-the-middle (человек посередине)**
    - Перехват и изменение коммуникации
    - Требует шифрования для защиты
5. **Privilege escalation (повышение привилегий)**
    - Эксплуатация уязвимостей для получения root
6. **Ransomware (программы-вымогатели)**
    - Шифрование данных с требованием выкупа
7. **Zero-day attacks (атаки нулевого дня)**
    - Эксплуатация неизвестных уязвимостей

Механизмы защиты

Компоненты:

- **Изоляция на уровне ОС**
    - Разделение процессов и пользователей
- **Песочница (Sandboxing)**
    - Изоляция ненадежных приложений
    - Ограничение доступа к ресурсам
- **Виртуализация**
    - Изоляция через виртуальные машины
- **Системы обнаружения вторжений (IDS)**
    - Мониторинг подозрительной активности
- **Принуждение контроля доступа**
    - Автоматическая проверка прав
- **Обязательные политики безопасности**
    - SELinux, AppArmor

Политики безопасности

Типичные политики включают:

- **Правила паролей**
    - Минимальная длина, сложность
    - Регулярная смена
    - История паролей
- **Права доступа и роли**
    - Кто может делать что
- **Сетевые ограничения**
    - Firewall правила
    - Белые/черные списки IP
- **Политика установки ПО**
    - Только утвержденное ПО
    - Цифровые подписи
- **Правила защиты данных**
    - Шифрование чувствительных данных
    - Контроль передачи данных
- **Требования к резервному копированию и восстановлению**
    - Регулярные бэкапы
    - Тестирование восстановления

Конспект 6: Продвинутые темы и обзор (Week 15)

Виртуализация

Что такое виртуализация?

Виртуализация — создание виртуальной (а не физической) версии вычислительных ресурсов: аппаратных платформ, операционных систем, хранилищ или сетей.

Ключевая идея: один физический компьютер может хостить несколько виртуальных машин (VM), каждая со своей гостевой ОС, работающих изолированно.

Преимущества:

- Совместное использование ресурсов
- Изоляция окружений
- Гибкость
- Упрощенное управление

Типы виртуализации

1. **Аппаратная (полная) виртуализация**
    - Полная эмуляция физического оборудования
    - Каждая VM запускает полноценную ОС
    - Гостевая ОС не знает о виртуализации
2. **Паравиртуализация**
    - Гостевая ОС знает о виртуализированной среде
    - Лучшая производительность через специальные API
    - Требует модификации гостевой ОС
3. **Виртуализация хранилища**
    - Объединение физических хранилищ в виртуальные единицы
4. **Виртуализация сети**
    - Создание виртуальных сетей, подсетей, сетевых устройств
5. **Виртуализация рабочих столов/приложений**
    - Изоляция рабочих столов или приложений для распространения и безопасности

Гипервизоры

Гипервизор — программное обеспечение, управляющее виртуальными машинами.

Два основных типа:

Type 1 (Bare-metal) — Гипервизор на голом железе:

- Запускается непосредственно на оборудовании
- Примеры: VMware ESXi, Microsoft Hyper-V, Xen, KVM (в режиме bare-metal)
- Высокая эффективность
- Используется в дата-центрах

Type 2 (Hosted) — Хостируемый гипервизор:

- Запускается поверх существующей хост-ОС
- Примеры: VirtualBox, VMware Workstation
- Проще в установке
- Больше накладных расходов и меньше производительности/изоляции

Виртуальные машины: плюсы и минусы

Преимущества:

- **Сильная изоляция** — каждая VM независима
- **Полная среда ОС** — поддерживает любой софтверный стек

- **Снимки, миграция, клонирование** — легкие бэкапы и масштабирование
- **Контроль ресурсов** — выделение CPU, памяти, I/O для каждой VM

Недостатки:

- **Большие накладные расходы** на ресурсы (RAM, диск, CPU)
- **Медленный запуск и выключение** по сравнению с легковесными окружениями
- **Менее эффективны** для множества мелких сервисов или микросервисов
- **Накладные расходы** на хранение и управление в масштабе

Облачные операционные системы

Что такое Cloud OS?

"Cloud OS" — слой программного обеспечения и сервисов, управляющий виртуализированными ресурсами дата-центра в масштабе.

Функции:

- Планирование ресурсов (вычисления, хранение, сеть)
- Мультитенантность (многие пользователи безопасно разделяют инфраструктуру)
- Изоляция между арендаторами
- Динамическое выделение, масштабирование и оркестрация

Примеры: AWS, Google Cloud Platform, Microsoft Azure — концептуально каждый ведет себя как огромная ОС для дата-центра.

Ключевые характеристики Cloud OS

- **Автоматическое выделение и управление ресурсами**
  - Выделение вычислений, хранилища, сети по запросу
- **Эластичная масштабируемость**
  - Динамическое масштабирование вверх/вниз на основе спроса
- **Объединение и разделение ресурсов**
  - Разделяемые аппаратные ресурсы между множеством арендаторов
- **Изоляция мультитенантности**
  - Обеспечение безопасности и приватности между клиентами
- **Высокая доступность и отказоустойчивость**
  - Избыточные ресурсы, живая миграция, failover
- **Глобальное распределение**
  - Дата-центры в разных регионах
  - Управление ресурсами с учетом задержек

Сервисы Cloud OS

Облачная ОС предоставляет сервисы, аналогичные компонентам традиционной ОС, но в масштабе дата-центра:

Вычислительные сервисы (Compute):

- Виртуальные машины
- Bare-metal серверы
- Контейнеры
- Serverless функции (AWS Lambda, Google Cloud Functions)

Сервисы хранения (Storage):

- Блочное хранилище (EBS, Persistent Disks)

- Объектное хранилище (S3, Cloud Storage)
- Распределенные файловые системы

Сетевые сервисы (Networking):

- Виртуальные сети (VPC)
- Балансировщики нагрузки
- Firewalls, VPN
- CDN (Content Delivery Networks)

Identity & Access Management (IAM):

- Управление пользователями и ролями
- Разрешения, изоляция мультитенантности

Мониторинг и логирование:

- Health checks
- Триггеры масштабирования
- Аналитика использования
- Биллинг

Контейнеры

Что такое контейнеры?

Контейнеры — легковесные, изолированные экземпляры пользовательского пространства, разделяющие ядро хост-ОС.

Инкапсулируют:

- Код приложения
- Зависимости (библиотеки, runtime)
- Конфигурацию

Предоставляют:

- Легковесную виртуализацию на уровне ОС
- Быстрый запуск и остановку (секунды или меньше)
- Малый след хранения (образы)
- Консистентное окружение между разработкой и продакшеном

Контейнеры vs Виртуальные машины

Характеристика	Контейнеры	Виртуальные машины
Накладные расходы	Низкие (разделяют ядро хоста)	Высокие (каждая VM имеет полную ОС)
Время запуска	Очень быстрое (секунды или меньше)	Медленное (минуты)
Эффективность ресурсов	Высокая — много контейнеров на хост	Низкая — меньше VM на хост
Уровень изоляции	Изоляция пользовательского пространства, разделяемое ядро	Сильная изоляция, отдельные экземпляры ОС
Случаи использования	Микросервисы, масштабируемые веб-приложения, CI/CD	Легаси приложения, полный стек ОС, сильная изоляция

Docker — пример контейнерной технологии

Docker — популярная платформа контейнеризации.

Ключевые концепции:

- **Docker Image** — шаблон, определяющий приложение + зависимости
  - Неизменяемый
  - Слоистая файловая система
- **Docker Container** — запущенный экземпляр образа
  - Изолированная файловая система, сеть, пространство имен
- **Dockerfile** — скрипт для создания образа

dockerfile

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y python3
COPY app.py /app/
CMD ["python3", "/app/app.py"]
```

Возможности:

- **Port mapping** — связывание портов контейнера с хостом
- **Volume mounts** — постоянное хранение данных
- **Environment configuration** — переменные окружения
- **Портируемость** — "build once, run anywhere"

Оркестрация контейнеров

Проблема: управление множеством контейнеров на нескольких хостах вручную сложно.

Системы оркестрации:

- **Kubernetes (K8s)** — стандарт индустрии
- **Docker Swarm** — встроенная оркестрация Docker

Функции:

- Автоматическое планирование контейнеров
- Масштабирование (горизонтальное)
- Балансировка нагрузки
- Health checks и самовосстановление
- Service discovery — автоматическое обнаружение сервисов
- Rolling updates — обновление без простоя
- Failover — автоматическое восстановление при сбоях

Kubernetes предоставляет:

- **Pods** — группа из одного или нескольких контейнеров
- **Services** — стабильные endpoints для доступа к pods
- **Deployments** — декларативное управление приложениями
- **ConfigMaps & Secrets** — управление конфигурацией

Распределенные системы

Что такое распределенные системы?

Распределенная система — сеть независимых компьютеров/узлов, сотрудничающих для того, чтобы казаться единой связной системой.

Особенности:

- Разделяемые или распределенные данные и вычисления
- Узлы взаимодействуют через сообщения по сети
- Отказоустойчивость, избыточность, масштабируемость
- Прозрачность для конечных пользователей (система выглядит единой)

Вызовы распределенных систем

- **Конкуренция и синхронизация** между сетевыми узлами
- **Задержки и разделы сети**
  - Непредсказуемые задержки
  - Частичные отказы сети
- **Компромиссы между консистентностью и доступностью**
- **Отказоустойчивость**
  - Узлы могут выходить из строя независимо
  - Система должна продолжать работать
- **Распределенная координация и консенсус**
  - Обеспечение согласия между узлами
  - Примеры: выборы лидера, распределенные транзакции

Теорема CAP

CAP Theorem: в распределенной системе данных вы можете гарантировать **максимум два** из следующих трех свойств:

- **Consistency (C) — Консистентность**
  - Все узлы видят одни и те же данные одновременно
  - Чтение всегда возвращает последнюю запись
- **Availability (A) — Доступность**
  - Каждый запрос получает ответ (без ошибки) — без гарантии, что это самые свежие данные
- **Partition Tolerance (P) — Устойчивость к разделению**
  - Система продолжает работать несмотря на потерю сообщений или разделение сети

Импликация: поскольку разделение сети неизбежно в распределенных системах, реальные системы должны выбирать между **консистентностью (CP)** и **доступностью (AP)**.

Примеры:

- **CP системы:** банковские транзакции — консистентность критична
- **AP системы:** социальные сети — доступность важнее немедленной консистентности

Распределенное хранилище и файловые системы

Примеры:

- **HDFS (Hadoop Distributed File System)**
  - Для big data обработки
  - Оптимизирован для больших файлов
- **Ceph**
  - Унифицированное хранилище: объектное, блочное, файловое
  - Самовосстановление
- **Google File System (GFS / Colossus)**
  - Внутренняя FS Google
  - Основа для MapReduce и BigTable

Особенности:

- **Репликация данных** между узлами для избыточности
- **Шардирование и партиционирование** для масштабируемости
- **Отказоустойчивость** — сбои узлов не приводят к потере данных
- **Высокая пропускная способность** для распределенных рабочих нагрузок

Объединяем все вместе: реальная архитектура

Современный стек инфраструктуры часто комбинирует:

1. **Виртуализация на базе гипервизора или bare-metal** для изоляции
2. **Cloud OS / облачная платформа** для управления инфраструктурой и ресурсами
3. **Контейнеры + оркестрация** для развертывания приложений (микросервисы)
4. **Распределенные системы и распределенное хранилище** для масштабируемости и устойчивости
5. **Мониторинг, логирование и автоматизированные пайплайны развертывания (CI/CD)**

Этот слоистый стек поддерживает масштабируемые, поддерживаемые и устойчивые приложения.

Пример архитектуры:

```
Приложение (микросервисы в контейнерах)
          ↓
    Kubernetes (оркестрация)
          ↓
   Виртуальные машины или bare-metal серверы
          ↓
      Гипервизор / Cloud OS
          ↓
   Физические серверы в дата-центрах
          ↓
Распределенное хранилище (HDFS, Ceph)
```

Резюме и ключевые выводы

- **Виртуализация** позволяет множеству изолированных окружений на одном физическом хосте
- **Cloud OS** расширяет принципы ОС до масштаба дата-центра, предлагая объединенные ресурсы, масштабируемость и мультитенантность
- **Контейнеры** предоставляют легковесные, эффективные и портируемые окружения, идеальные для микросервисов и быстрого развертывания
- **Оркестрация контейнеров** автоматизирует развертывание, масштабирование и управление множеством контейнеров на нескольких хостах
- **Распределенные системы** масштабируют вычисления и хранение на множество машин, но требуют аккуратной обработки консистентности, доступности и отказов
- **Современные системы комбинируют все эти технологии** для создания масштабируемых, устойчивых и эффективных сервисов

Общие рекомендации для подготовки к экзамену:

1. **Понимайте концепции, а не только запоминайте** — важно понимать "почему" и "когда", а не только "что"
2. **Связывайте темы** — многие концепции взаимосвязаны (например, синхронизация связана с управлением процессами)
3. **Практикуйте алгоритмы** — попробуйте вручную выполнить алгоритмы планирования, замещения страниц
4. **Знайте компромиссы** — для каждой технологии понимайте плюсы и минусы
5. **Примеры из реальной жизни** — связывайте концепции с реальными системами (Linux, Windows,